

Word Embedding

Muhammad Yazid Supriadi, S.Kom, M.Kom
STT Terpadu Nurul Fikri

Tujuan Perkuliahan

- Memahami pentingnya word embedding di nlp
- Memahami konsep dari beberapa model word embedding
- Implementasi word embedding

- Dalam natural language processing, komputer tidak bisa langsung **paham** kata-kata seperti manusia.
- Word embeddings mengatasi tantangan ini dengan merepresentasikan kata-kata dalam ruang vektor yang kontinu, padat, dan berdimensi lebih rendah (dense vector), memungkinkan mesin untuk memahami makna, hubungan, dan konteks kata-kata dengan lebih efektif.
- Setiap word embedding dapat mengandung ratusan atau ribuan dimensi yang berisi berbagai "konteks".
- Tujuan dari word embedding adalah memetakan Kata-kata yang mirip makna atau konteks akan berada dekat satu sama lain di ruang vektor tersebut.

Word Embedding

	Place	Vehicle
House	10	0
Car	0	10
Eyes	0	0

- **ASCII (American Standard Code for Information Interchange)** adalah sistem encoding yang mengubah karakter menjadi angka.
- ASCII merupakan cara paling simple yang bisa digunakan untuk merepresentasikan teks.
- Terdapat beberapa problem dengan penerapan ASCII dalam representasi teks diantaranya ASCII tidak dapat menangkap makna dari sebuah kata. Sebagai contoh: “good” dan “great” punya arti sama tapi jika direpresentasikan dengan ASCII maka nilai vektornya berbeda.
- ASCII juga tidak dapat menggambarkan hubungan dari sebuah kata. Sebagai contoh “good” dan “bad”. Kedua kata tersebut artinya sangat berlawanan. ASCII akan tetapi mereka diwakili oleh vektor yang sangat berbeda dalam ASCII.

One-Hot Encoding adalah metode tradisional untuk merepresentasikan kata sebagai vektor numerik, di mana:

- Setiap kata direpresentasikan sebagai vektor
- Semua nilai = 0, kecuali satu indeks = 1
- Panjang vektor = jumlah kata dalam vocabulary

Ciri Utama:

- Sederhana dan mudah dipahami
- Tidak mengandung informasi makna (semantik)
- Dimensi tinggi jika vocabulary besar
- Tidak menangkap hubungan antar kata

One Hot Encoding

Kalimat:

"the cat sat on the mat"

Vocabulary:

[the, cat, sat, on, mat]

Representasi:

the $\rightarrow$ [1, 0, 0, 0, 0]

cat $\rightarrow$ [0, 1, 0, 0, 0]

sat $\rightarrow$ [0, 0, 1, 0, 0]

on $\rightarrow$ [0, 0, 0, 1, 0]

mat $\rightarrow$ [0, 0, 0, 0, 1]

Masalah One Hot Encoding

1. Dimensi Sangat Besar

Panjang vektor = jumlah vocabulary

Tidak efisien untuk penyimpanan dan komputasi

Contoh:

Vocabulary = 10.000 kata

Maka setiap kata direpresentasikan sebagai vektor berukuran **10.000**

2. Tidak Efisien (Wasteful)

Hanya **1 elemen bernilai 1**, sisanya 0. sehingga banyak ruang terbuang karena mayoritas bernilai nol

Contoh:

“hello” $\rightarrow$ [0, 0, 0, ..., 1, ..., 0]

3. Sparse Vector (Jarang)

Vektor sangat jarang (sparse)

Mayoritas elemen = 0

Masalah sparse vector menyebabkan sulit dilatih oleh model serta rentan terhadap **curse of dimensionality**

4. Tidak Menangkap Hubungan Antar Kata

Setiap kata dianggap **independen**

Tidak ada informasi semantik

Contoh:

“king” $\neq$ “ruler” (padahal maknanya mirip)

5. Tidak Bisa Generalisasi

Model tidak bisa memahami hubungan makna sebagai contoh:

Training:

“The king conquered the land”

Testing:

“The ruler conquered the ...”

Model tidak bisa menebak “land” karena tidak tahu bahwa **king** $\approx$ **ruler**

4. Tidak Menangkap Hubungan Antar Kata

Setiap kata dianggap **independen**

Tidak ada informasi semantik

Contoh:

“king” $\neq$ “ruler” (padahal maknanya mirip)

5. Tidak Bisa Generalisasi

Model tidak bisa memahami hubungan makna sebagai contoh:

Training:

“The king conquered the land”

Testing:

“The ruler conquered the ...”

Model tidak bisa menebak “land” karena tidak tahu bahwa **king** $\approx$ **ruler**

4. Tidak Menangkap Hubungan Antar Kata

Setiap kata dianggap **independen**

Tidak ada informasi semantik

Contoh:

“king” $\neq$ “ruler” (padahal maknanya mirip)

5. Tidak Bisa Generalisasi

Model tidak bisa memahami hubungan makna sebagai contoh:

Training:

“The king conquered the land”

Testing:

“The ruler conquered the ...”

Model tidak bisa menebak “land” karena tidak tahu bahwa **king** $\approx$ **ruler**

Mengapa Word Embedding diperlukan.?

Word embeddings mengubah kata-kata menjadi vektor padat berdimensi rendah yang mempertahankan hubungan semantik dan makna kontekstual. Berbeda dengan BoW dan TF-IDF, yang hanya mempertimbangkan kemunculan kata, word embeddings menangkap makna dan hubungan antar kata.

Dengan adanya word embedding dapat dilakukan:

- 1. Menangkap sinonim dan kata-kata yang serupa**

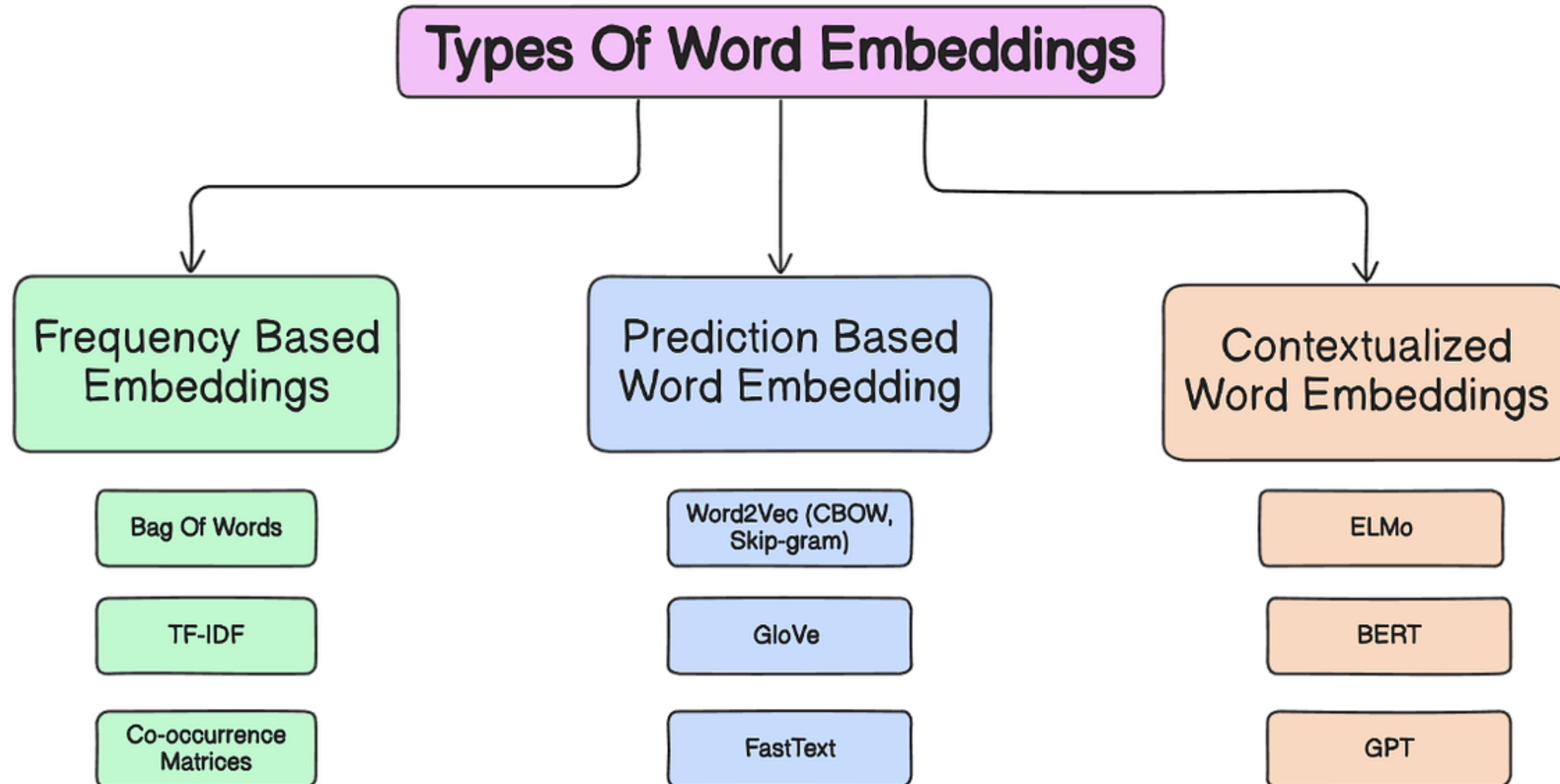
Sebagai contoh : Vektor untuk “happy” dan “joyful” secara matematis akan berada dekat dalam ruang berdimensi tinggi.

- 2. Memahami Analogi dan hubungan antar kata**

Sebagai contoh : Hubungan antara “king” dan “queen” mirip dengan hubungan antara “man” dan “woman.”

- 3. Mempertahankan makna kontekstual**

Metode NLP tradisional memperlakukan kata secara terpisah, artinya kata "bank" dalam “river bank” dan “bank account” akan diperlakukan sama. Word embeddings menangkap konteks, sehingga representasi “bank” dalam konteks yang berbeda akan berbeda.



Word2Vec (Word to Vector)

Word2Vec adalah model yang dikembangkan oleh Google pada tahun 2013 untuk menghasilkan representasi kata dalam bentuk **dense vector** yang mampu menangkap **makna dan konteks**.

Konsep utama dari word2vec:

- Berbasis deep learning (predictive model)
- Bersifat unsupervised (tidak butuh label)
- Dilatih menggunakan corpus teks besar
- Menghasilkan representasi kata dalam bentuk vektor kontinu
- Kata dengan makna mirip → vektor mirip

Karakteristik dari word2vec:

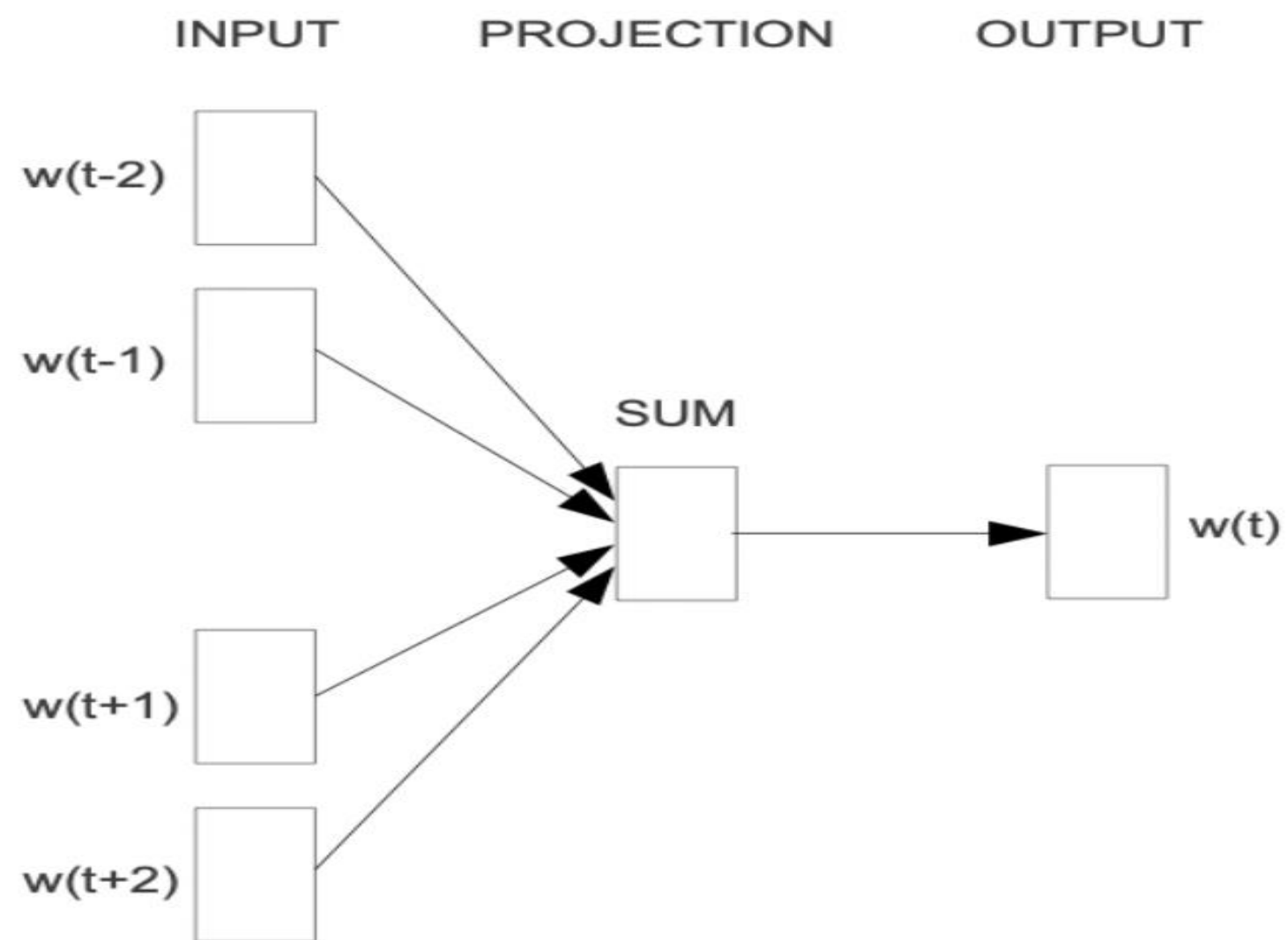
- Membentuk **vocabulary** dari corpus
- Setiap kata direpresentasikan sebagai **embedding vector**
- Dimensi vektor bisa ditentukan (misal: 100, 300)

Word2Vec (Word to Vector)

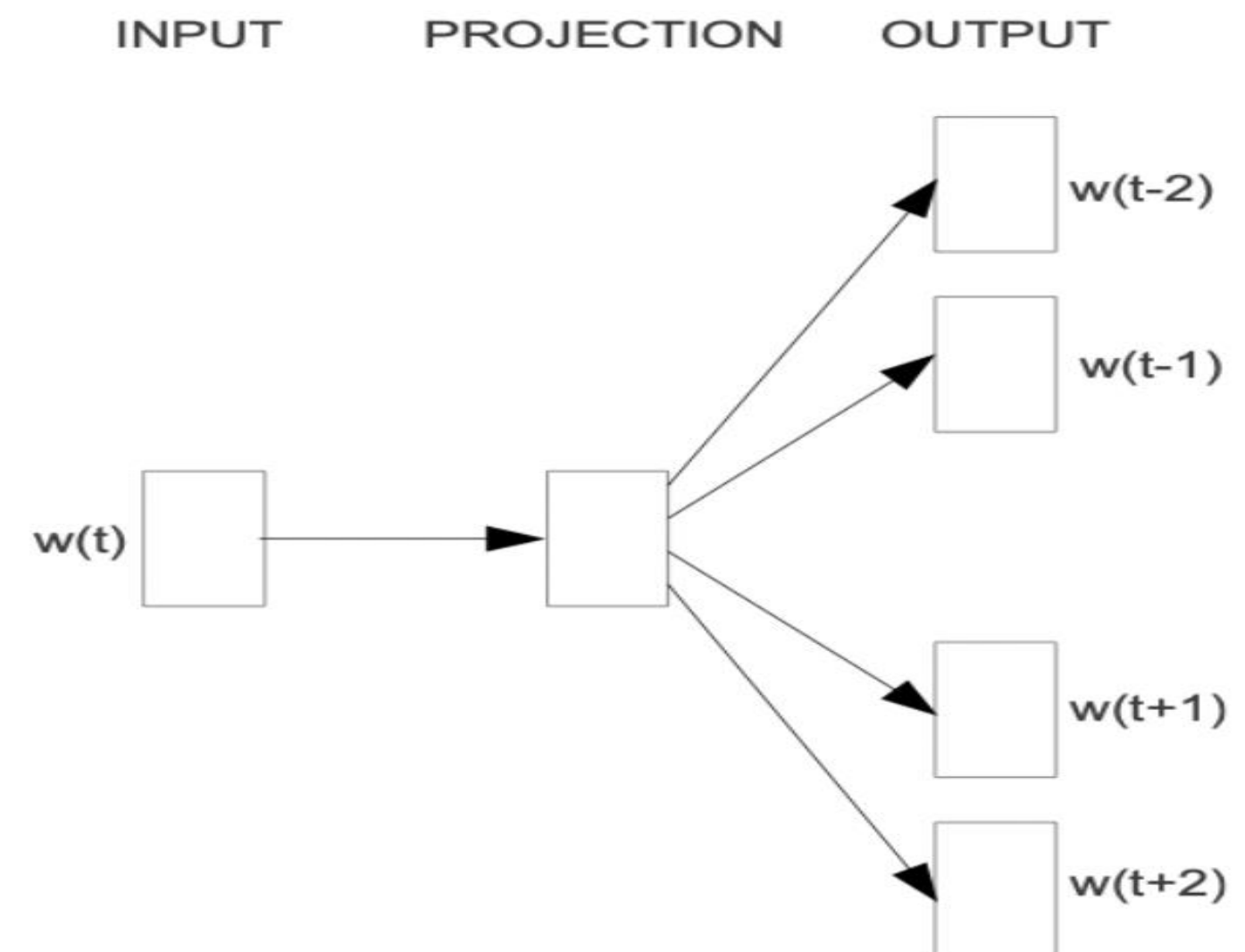
Word2Vec memiliki 2 model utama yaitu:

- **CBOW (Continuous Bag of Words)**
- **Skip-Gram**

Arsitektur Word2vec hanya terdiri dari 3 layer yaitu **Input**, **Projection (Hidden Layer)**, dan **Output**



CBOW



Skip-gram

Skip-Gram

Skip-gram memprediksi kata-kata di sekitarnya berdasarkan satu kata target. Alih-alih memprediksi satu kata dari banyak input, ia memprediksi banyak kata berdasarkan satu kata.

Cara kerja skipgram:

1. Ambil satu kata sebagai target
2. Tentukan window size
3. Ambil kata di sekitar sebagai context
4. Buat pasangan (target → context)
5. Latih model untuk memprediksi context dari target

Skip-gram adalah **shallow neural network** (hanya 1 hidden layer) dengan struktur berikut:

- **Input Layer:** One-hot vector dari kata target (ukuran = ukuran vocabulary, misal 10.000 kata).
- **Hidden Layer (Embedding Layer):** Proyeksi ke dimensi rendah (biasanya 100–300 dimensi). Ini yang menjadi **word embedding** akhir.
- **Output Layer:** Softmax untuk memprediksi banyak konteks (ukuran = vocabulary).

$$P(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{u}_{w_{t+j}}^T \mathbf{v}_{w_t})}{\sum_{w=1}^V \exp(\mathbf{u}_w^T \mathbf{v}_{w_t})}$$

Di mana:

$\mathbf{v}_{w_t}$ =vektor input (embedding) kata target

$\mathbf{u}_w$ =vektor output kata konteks

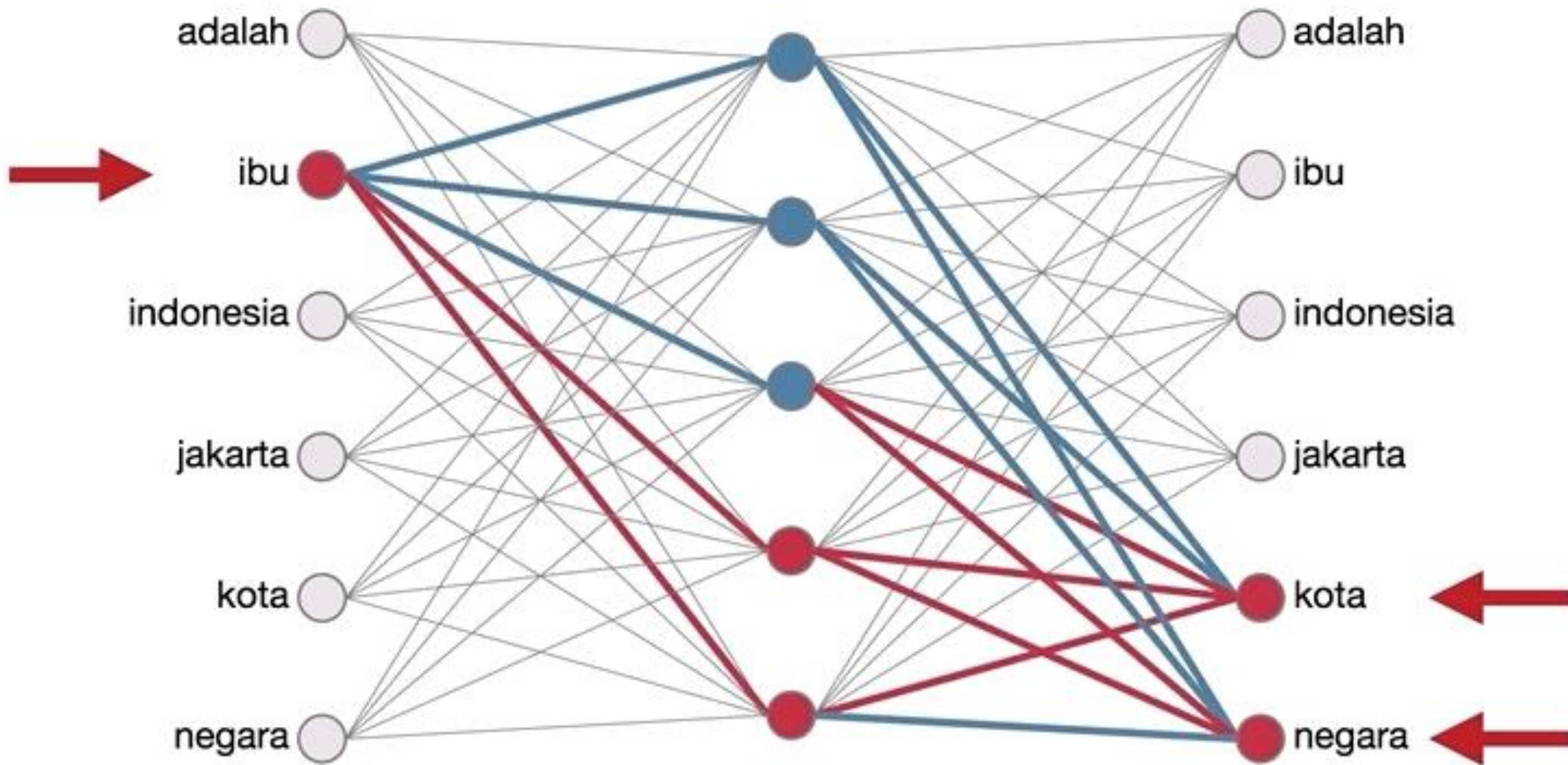
Proyeksi Skip-Gram

Sebagai contoh terdapat sebuah kalimat “Ibu kota Negara Indonesia Adalah Jakarta”.
Dicoba training dengan window size = 2

Ibu kota Negara Indonesia adalah Jakarta

input target

Proyeksi Skip-Gram

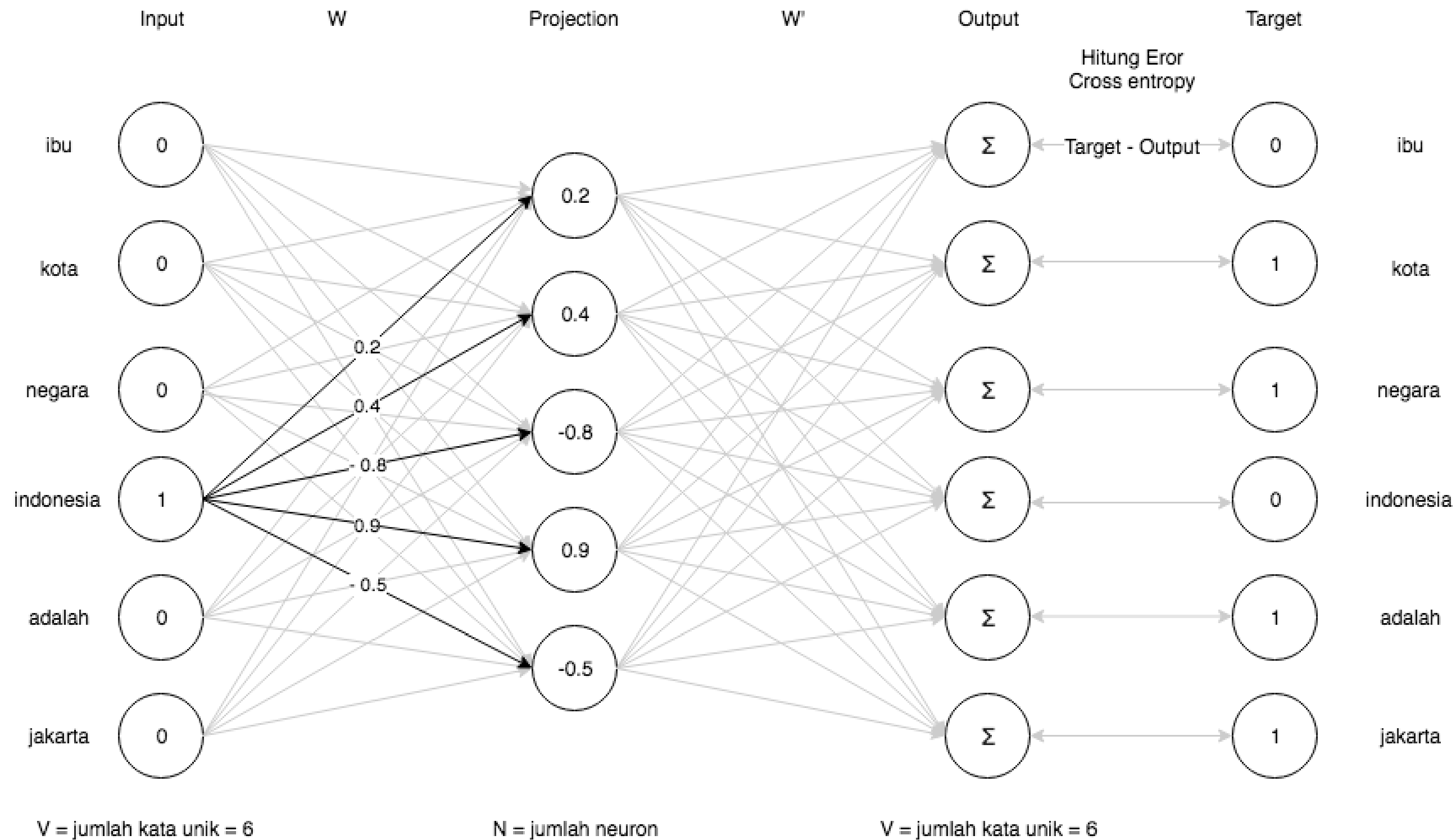


Data yang diinput merupakan *one-hot encoded vector* sehingga membentuk data sebagai berikut:

Ibu	[1,0,0,0,0,0]
Kota	[0,1,0,0,0,0]
Negara	[0,0,1,0,0,0]
Indonesia	[0,0,0,1,0,0]
Adalah	[0,0,0,0,1,0]
jakarta	[0,0,0,0,0,1]

Proyeksi Skip-Gram

Apabila input = Indonesia, maka proyeksi dari skip-gram sebagai berikut:



Beberapa Keuntungan dengan menggunakan Skip-gram:

- Bekerja lebih baik pada kata-kata langka karena mempelajari representasi kata secara individual.
- Lebih efektif untuk dataset besar karena menangkap hubungan semantik yang lebih rinci.
- Menghasilkan embedding yang lebih representatif untuk kata penting
- Mampu mempelajari **relasi semantik yang lebih halus**

Continuous Bag of Word (CBOW)

Continuous Bag of Word (CBOW) adalah kebalikan dari Word2vec skip-gram. Tujuannya adalah untuk memprediksi kata (output) ketika diberikan konteks disekitar kata tersebut (input).

Model ini melihat “tas” (bag) dari kata-kata konteks, lalu memprediksi kata yang paling mungkin muncul di tengahnya. Kata-kata konteks dianggap tidak berurutan (bag of words), hanya rata-rata vektornya yang dipakai.

Cara kerja CBOW:

1. **Pilih Window** :Ambil kata target dan kata-kata di sekitarnya.
2. **Buat Input** : Setiap kata konteks diubah menjadi one-hot vector.
3. **Lookup Embedding** : Setiap one-hot dikalikan dengan matriks embedding → dapat vektor dense.
4. **Averaging (Bag of Words)** : Semua vektor konteks **dirata-ratakan** menjadi satu vektor konteks.
5. **Prediksi Target** : Vektor rata-rata tersebut dikalikan dengan matriks output → skor untuk setiap kata di vocabulary.

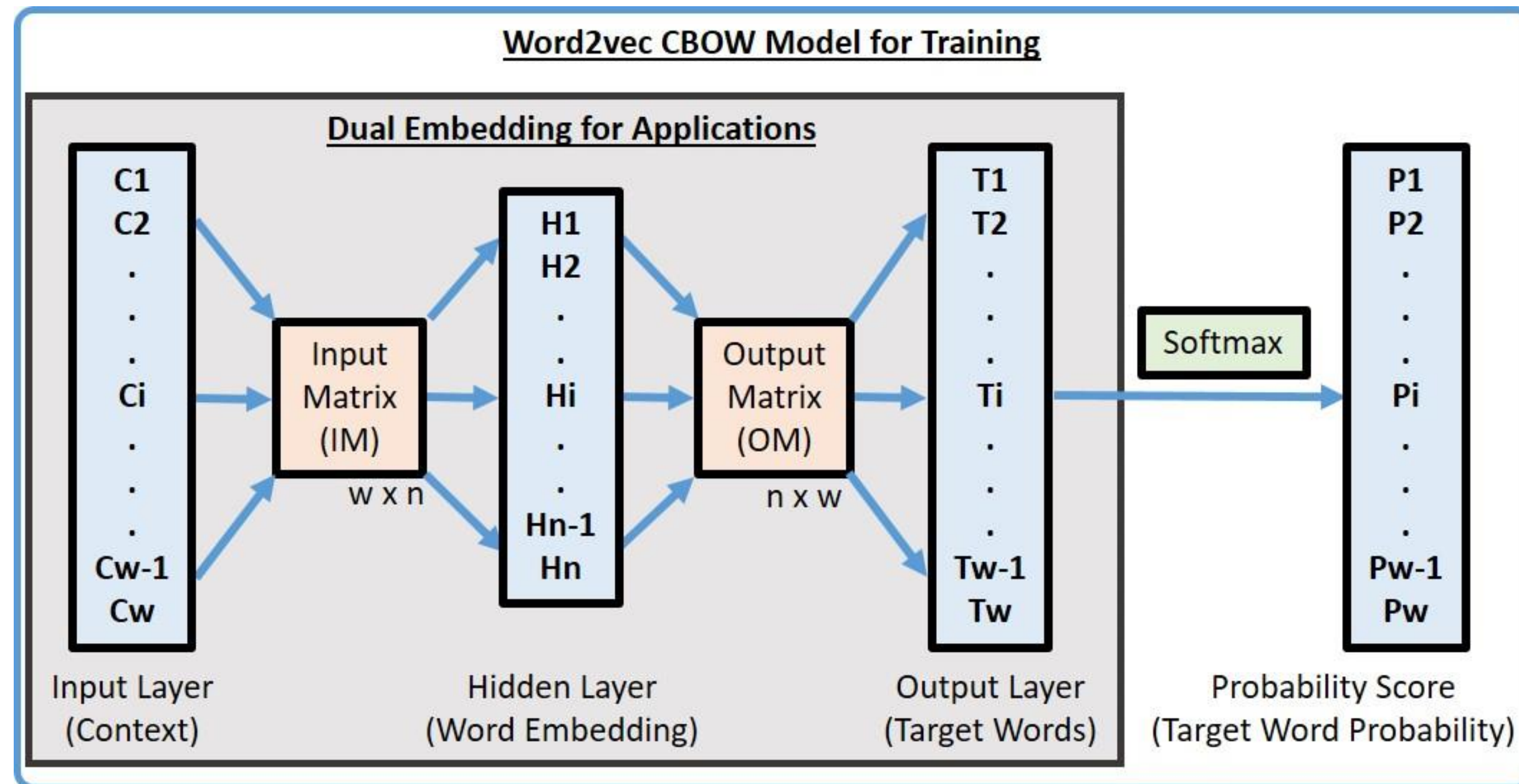
Continuous Bag of Word (CBOW)

CBOW juga menggunakan **shallow neural network** (1 hidden layer):

Input Layer: Banyak one-hot vector dari kata-kata konteks (jumlah = window size $\times$ 2).

Hidden Layer (Embedding Layer): Rata-rata (average) dari vektor embedding semua konteks.

Output Layer: Softmax untuk memprediksi 1 kata target.



Continuous Bag of Word (CBOW)

Langkah utama perhitungan CBOW:

1. Vektor konteks di-average:

$$\mathbf{h} = \frac{1}{C} \sum_{j=-c, j \neq 0}^c \mathbf{v}_{w_{t+j}}$$

2. Probabilitas prediksi kata target:

$$P(w_{t'} | w_{t-c} \dots w_{t+c}) = \frac{\exp(\mathbf{u}_{w_{t'}}^T \mathbf{h})}{\sum_{w=1}^V \exp(\mathbf{u}_w^T \mathbf{h})}$$

Di mana:

$\mathbf{v}$ = vektor input (context embeddings)

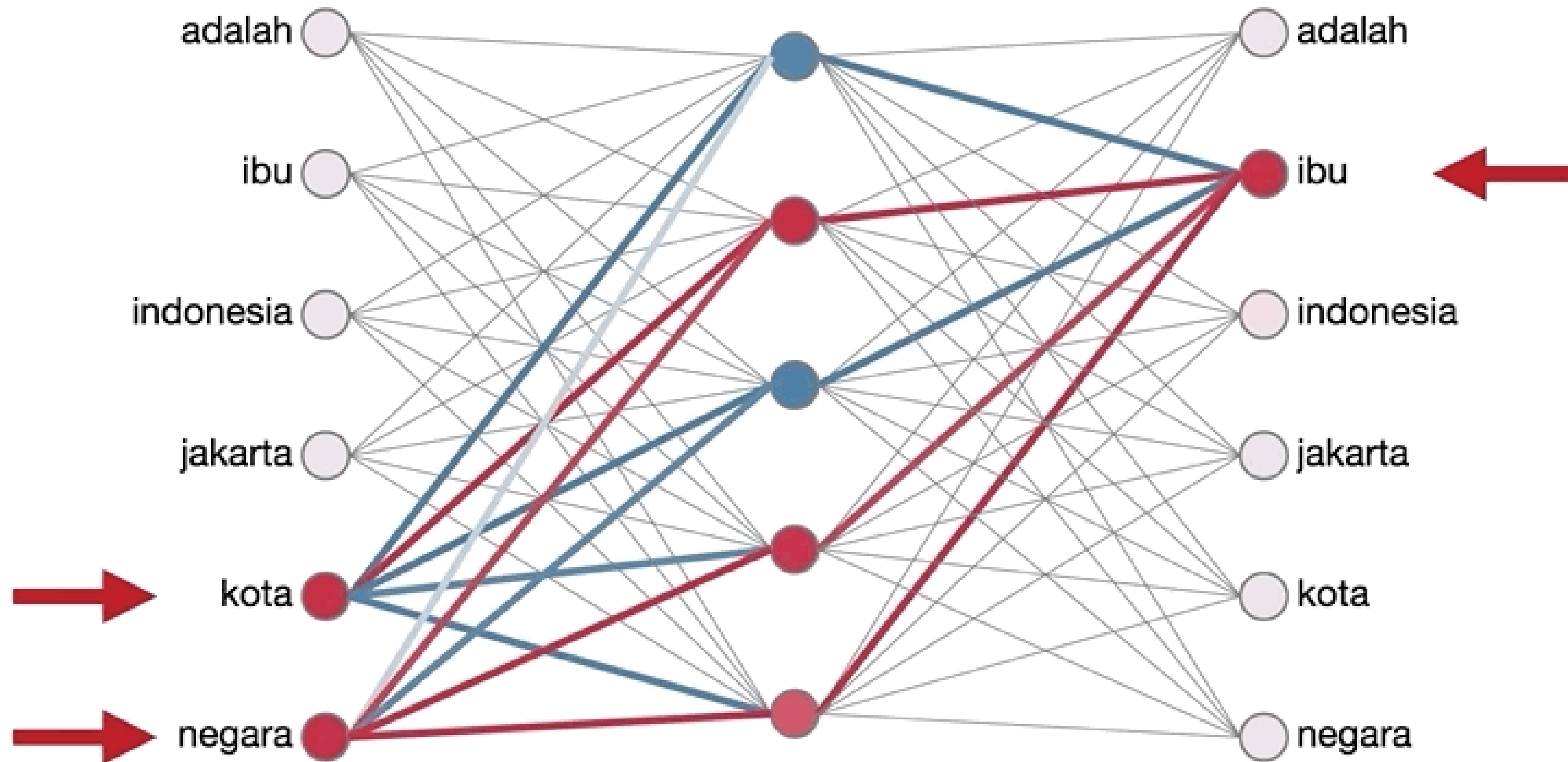
$\mathbf{u}$ = vektor output (prediction embeddings)

$\mathbf{h}$ = hidden representation (rata-rata konteks)

Sebagai contoh terdapat sebuah kalimat “Ibu kota Negara Indonesia Adalah Jakarta”.
Dicoba training dengan window size = 2

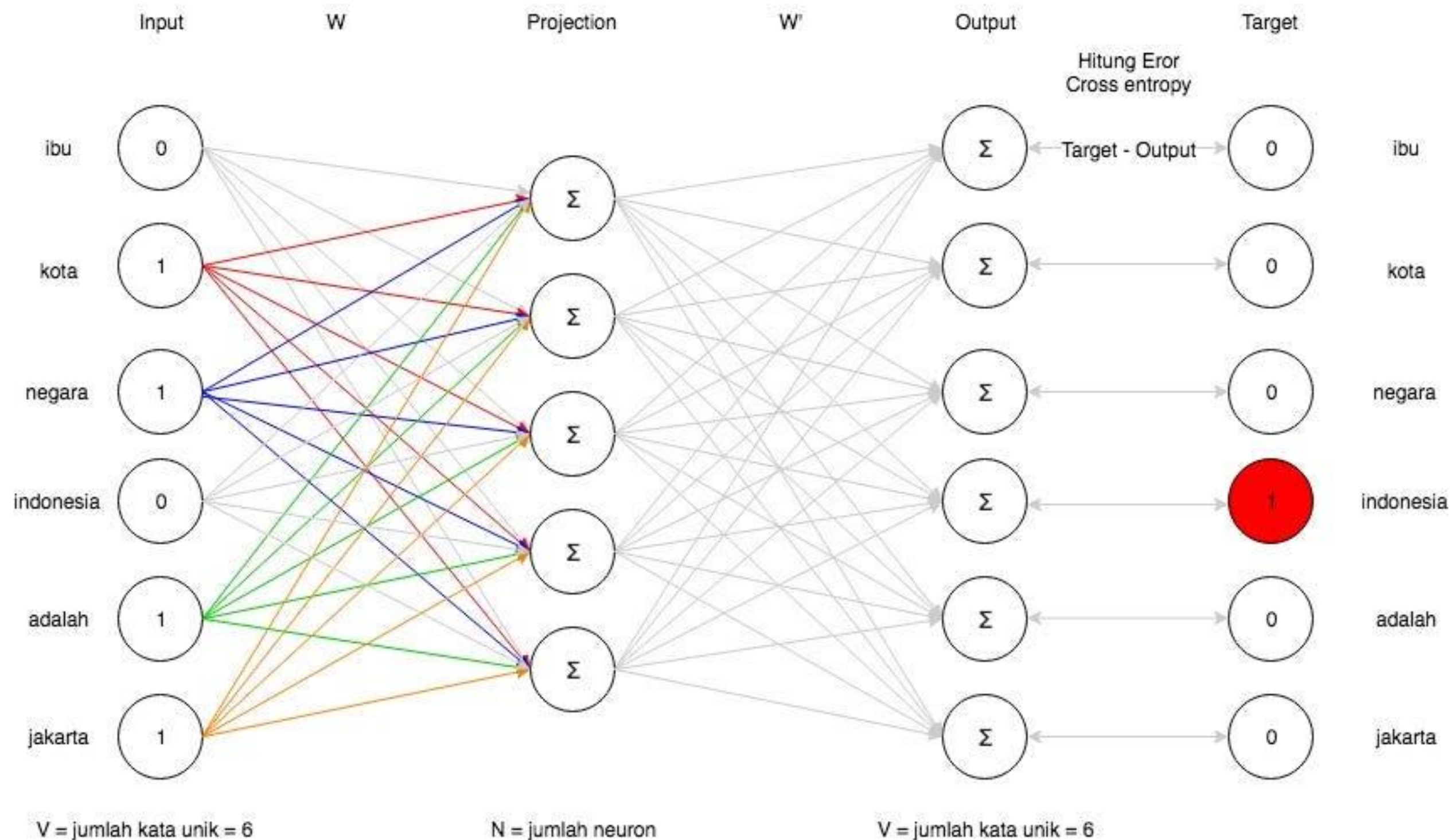


Proyeksi CBOW



Proyeksi Skip-Gram

Data input CBOW berbentuk *n-hot encoded vector*. Ketika proses training berjalan, kata yang sedang menjadi inputan akan bernilai 1 sedangkan kata yang lainnya akan bernilai 0. Karena targetnya hanya 1 kata, maka target output akan berbentuk *one-hot encoded vector*. Apabila input = 'indonesia' maka proyeksinya sebagai berikut:



Continous Bag of Word (CBOW)

Beberapa Keuntungan dengan menggunakan Continous Bag of Word (CBOW):

- Pelatihan lebih cepat karena memprediksi satu kata dari beberapa input.
- Bekerja dengan baik untuk dataset kecil.
- Representasi vektor kata yang halus karena rata-rata kata konteks.
- Menghasilkan embedding yang baik untuk kata yang sering muncul
- Hemat komputasi

Continuous Bag of Word (CBOW) vs Skip-gram

Aspek	CBOW	Skip-gram
Input	Banyak kata konteks	1 kata target
Output	1 kata target	Banyak kata konteks
Kecepatan training	Lebih cepat	Lebih lambat
Cocok untuk	Kata yang sering muncul (frequent words)	Kata langka / rare words
Kualitas embedding	Bagus untuk tugas umum	Lebih baik untuk analogi & rare words
Window size	Biasanya kecil	Bisa lebih besar
Penggunaan	Korpus besar, training cepat	Ketika data terbatas atau butuh rare words

Meskipun Word2Vec sangat efektif, ia memiliki beberapa keterbatasan:

- **Mengabaikan Urutan Kata** – Ini memperlakukan kata-kata sebagai entitas terpisah tanpa mempertimbangkan struktur kalimat.
- **Memerlukan Dataset Besar** – Untuk menghasilkan embedding kata yang bermakna, Word2Vec membutuhkan jumlah data teks yang cukup besar.
- **Kesulitan dengan Kata di Luar Kosa Kata** – Jika sebuah kata tidak ada dalam data pelatihan, model tidak bisa menghasilkan vektor untuk kata tersebut.
- **Pemahaman Konteks Terbatas** – Word2Vec tidak menangani polisemi dengan baik, artinya kata-kata dengan banyak makna mungkin memiliki satu representasi vektor.

GloVe (Global Vectors for Word Representation) adalah algoritma untuk membuat *word embeddings*. GloVe merepresentasikan kata dalam bentuk vektor numerik berdimensi rendah. GloVe dikembangkan oleh Jeffrey Pennington, Richard Socher, dan Christopher Manning dari Stanford University pada tahun 2014.

GloVe menghasilkan representasi vektor kata dengan memanfaatkan **statistik ko-kemunculan global** dari seluruh corpus.

Ide utama dari GloVe: kata-kata yang sering muncul bersama dalam konteks yang sama cenderung memiliki makna yang mirip, dan hubungan ini bisa dikodekan ke dalam vektor matematika.

Count-Based



Membangun matriks co-occurrence
dari seluruh corpus training

Statistik Global



Memanfaatkan frekuensi
kemunculan bersama di seluruh
dokumen

Vektor Bermakna



Hubungan semantik dan sintaksis
tertangkap dalam geometri vektor

- Sebelum GloVe, ada dua pendekatan utama:

Metode	Kelebihan	Kekurangan
Count-based (LSA, co-occurrence matrix)	Memanfaatkan statistik global	Tidak menangkap konteks lokal dengan baik
Prediction-based (Word2Vec)	Menangkap konteks lokal dengan baik	Mengabaikan statistik global corpus

- GloVe **menggabungkan keduanya** — memanfaatkan statistik global sekaligus menangkap pola konteks lokal.

Dimensi Vector Glove

GloVe biasanya tersedia dalam dimensi:

- **50d** — ringan, cocok untuk tugas sederhana
- **100d** — keseimbangan umum
- **200d** — lebih kaya representasi
- **300d** — paling detail, dilatih pada Common Crawl (840B token)

Stanford menyediakan pre-trained GloVe vectors yang bisa langsung digunakan.

Pre-trained dapat diakses di: [GloVe: Global Vectors for Word Representation](#)

Cara Kerja Glove:

1. Kumpulkan kata & konteks

Ambil semua kata dalam corpus, lalu lihat kata mana sering muncul **berdekatan**

2. Buat co-occurrence matrix

hitung berapa sering kata i muncul dekat kata j

3. Transform ke log

Ubah nilai frekuensi menjadi dengan rumus : $\log(X_{ij})$

4. Latih vector kata

Model mencari vektor tiap kata supaya:

$$W_i^t W_j \approx \log(X_{ij})$$

kata yang sering muncul bersama $\rightarrow$ vektornya mirip

5. Hasil Akhir

Setiap kata punya **vector embedding** yang menangkap makna

- Menangkap global statistics : Glove lebih baik dalam word similarity dan word analogy.
- Efisien untuk corpus besar.
- Linear substructures yang sangat jelas (salah satu yang terbaik di antara static embedding).
- Cocok untuk tugas seperti sentiment analysis, named entity recognition, dan machine translation preprocessing.
- Pre-trained model resmi tersedia dalam berbagai ukuran (50d, 100d, 200d, 300d) dan corpus (Wikipedia + Gigaword, Common Crawl, Twitter).

Kekurangan GloVe

- Masih static embedding → satu kata = satu vektor (tidak kontekstual seperti BERT).
- Tidak bisa handle OOV yang artinya Kata yang tidak ada di corpus training tidak memiliki representasi..
- Tidak dapat menangani kata dengan morfologi kaya kata seperti 'run', 'running', 'runner', 'ran' diperlakukan sebagai entitas terpisah meski berbagi akar makna yang sama.
- Membutuhkan corpus yang sangat besar agar matriks co-occurrence akurat.
- Training memerlukan memori besar.

- **FastText** adalah library dan metode *word embedding* yang dikembangkan oleh **Facebook AI Research (FAIR)** pada tahun 2016, diperkenalkan oleh Bojanowski, Grave, Joulin, dan Mikolov. FastText merupakan **evolusi dari Word2Vec** dengan satu inovasi kunci yang mengubah segalanya: **representasi kata berbasis subword (karakter n-gram)**.
- Ide utama: daripada memperlakukan setiap kata sebagai unit atomik, FastText memecah kata menjadi potongan-potongan karakter kecil.

GloVe dan Word2Vec punya dua kelemahan fundamental:

- 1. Out-of-Vocabulary (OOV)** — Kata yang tidak ada di corpus training tidak punya representasi sama sekali.
- 2. Morfologi diabaikan** — Kata *run, running, runner, ran* dianggap entitas yang sepenuhnya berbeda, padahal berbagi akar makna yang sama.

Ini sangat bermasalah untuk bahasa-bahasa **morfologis kaya** seperti:

- Bahasa Indonesia: *makan* → *memakan, dimakan, termakan, makanan*
- Bahasa Arab, Turki, Finlandia, dll.

Dengan adanya FastText dapat menyelesaikan kedua masalah tersebut sekaligus.

Contoh: kata "where" dengan $n = 3-6$

<where>



<wh

whe

her

ere

re>

<whe

wher

here

ere>

<wher

where

her>

<where>

Vektor kata = jumlah vektor semua n-gramnya:

$$v(\text{where}) = v(\text{<wh}) + v(\text{whe}) + v(\text{her}) + v(\text{ere}) + v(\text{re>}) + v(\text{<where>})$$

✗ GloVe / Word2Vec

Kata "unhappiness" tidak ada di vocabulary:

ERROR: 'unhappiness' not in vocabulary

- Sistem tidak bisa memproses kata ini
- Harus dihapus atau diganti <UNK>
- Kehilangan informasi penting
- Masalah besar untuk bahasa kaya morfologi

✓ FastText

Kata "unhappiness" → dipecah jadi n-gram:

<un	unh	nha	hap
app	ppi	pin	ine
nes	ess	ss>	

- Setiap n-gram punya vektor terlatih
- Kata OOV tetap mendapat representasi
- Tidak kehilangan informasi morfologis
- Sangat berguna untuk Bahasa Indonesia!

Arsitektur Classifier FastText (Supervised)



Word Embedding Comparison

Aspek	Word2Vec	GloVe	FastText
Pendekatan	Neural prediktif	Matrix factorization	Subword + prediktif
Unit representasi	Kata utuh	Kata utuh	Subword (n-gram)
OOV handling	Tidak bisa	Tidak bisa	Bisa
Morfologi	Diabaikan	Diabaikan	Ditangkap
Statistik global	-	Utama	-
Kecepatan training	Cepat	Cepat	Lebih lambat
Ukuran model	Kecil	Kecil	Lebih besar
Bahasa morfologis	Kurang	Kurang	Sangat cocok
Klasifikasi built-in	-	-	Ada
Pre-trained 157 bahasa	Sebagian	6B–840B token	Lengkap
Cocok untuk B. Indonesia	-	-	Terbaik

Thank you guys!